

1-Question: Why is it recommended to explicitly assign values to enum members in some cases?

A: Explicitly assigning values ensures consistency, especially when the enum is mapped to external systems like a database or an API. If you rely on auto-numbering, adding a new enum member in the middle later on will shift all subsequent values, corrupting existing data.

2-Question: What happens if you assign a value to an enum member that exceeds the underlying type's range?

A: The compiler will throw an error. For example, if the underlying type is `byte` (which holds 0 to 255), assigning 300 to an enum member will fail to compile.

3-Question: What is the purpose of the `virtual` keyword when used with properties?

A: The `virtual` keyword allows derived (child) classes to `override` the behavior of that property, providing a custom implementation for getting or setting the value while maintaining the same signature.

4-Question: Why can't you override a sealed property or method?

A: The `sealed` keyword strictly prevents further inheritance or modification. Once a method/property is sealed, C# guarantees its behavior cannot be changed by any derived class down the hierarchy.

5-Question: What is the key difference between static and object members?

A:

Static Members: Belong to the class itself. They are shared across all instances and can be called directly using the class name (e.g., `Utility.Method()`).

Object (Instance) Members: Belong to a specific object created from the class. They require instantiating the class with the `new` keyword to be accessed.

6-Question: Can you overload all operators in C#? Explain why or why not.

A: No, not all operators can be overloaded. Operators like `+`, `-`, `*`, `==` can be overloaded. However, logical operators like `&&`, `||`, and member access `.` cannot be overloaded to preserve fundamental language behavior and prevent unpredictable logical execution.

7-Question: When should you consider changing the underlying type of an enum?

A: You should change it (e.g., to `byte` or `short`) when memory optimization is critical, especially if you are creating large arrays of enums or sending data over a network, and you know the enum values will never exceed the smaller type's capacity.

8-Question: Why can't a static class have instance constructors?

A: Because a static class cannot be instantiated using the `new` keyword. An instance constructor's only job is to initialize a specific object in memory, which is impossible for a static class.

9-Question: What are the advantages of using `Enum.TryParse` over direct parsing with `int.Parse`?

A: `Enum.TryParse` is safer and prevents runtime crashes (Exceptions). If the user inputs an invalid string, `int.Parse` will throw a `FormatException` and crash the program, whereas `TryParse` handles it gracefully, returning `false`.

10-Question: What is the difference between overriding `Equals` and `==` for object comparison in C# struct and class?

A:

Structs (Value Types): `Equals` compares the actual values inside the struct. `==` does not work by default unless explicitly overloaded.

Classes (Reference Types): Both `Equals` and `==` default to comparing "Reference Equality" (memory addresses). Overriding `Equals` allows you to compare the actual object data instead of their memory locations.

11-Question: Why is overriding `ToString` beneficial when working with custom classes?

A: By default, `ToString()` prints the namespace and class name. Overriding it allows you to output a meaningful, human-readable string representation of the object's actual state/data, making debugging much easier.

12-Question: Can generics be constrained to specific types in C#? Provide an example.

A: Yes, constraints can be applied using the `where` keyword to restrict what types are allowed. Example: `public void MyMethod<T>(T item) where T : class` restricts `T` to only be reference types.

13-Question: What are the key differences between generic methods and generic classes?

A:

Generic Class: The type parameter `<T>` is defined at the class level and is available to all members within the class (e.g., `List<T>`).

Generic Method: The type parameter `<T>` is defined only for that specific method. The class itself doesn't need to be generic.

14-Question: Why might using a generic swap method be preferable to implementing custom methods for each type?

A: It promotes "Code Reusability". Instead of writing 10 different Swap methods for `int`, `double`, `string`, etc., a single generic method handles all data types safely and efficiently without code duplication.

15-Question: How can overriding `Equals` for the `Department` class improve the accuracy of searches?

A: When searching an array of `Employees`, the compiler needs to check if the `Employee`'s `Department` matches the target. Without overriding `Equals` in `Department`, it only checks if they share the same memory reference. Overriding it ensures it checks if the ID and Name match, yielding accurate search results.

16-Question: Why is `==` not implemented by default for structs?

A: Structs can contain complex fields (like strings or references). Comparing all these fields automatically could be performance-heavy. `C#` forces the developer to manually overload `==` if they truly need this behavior, ensuring performance transparency.

Part 02

1- LinkedIn article about class types?

A: *Understanding C# Class Types: The Foundation of OOP!* 🚀 Did you know `C#` offers several specialized class types to architect your software perfectly?

Standard Classes: The blueprint for standard objects.

Static Classes: Cannot be instantiated; used for global utilities and helpers (like `Math`).

Sealed Classes: The end of the line! Cannot be inherited, ensuring security and performance.

Abstract Classes: Master blueprints containing base logic, but require derived classes to finish the job.

Partial Classes: Allows splitting a single class across multiple files, great for team collaboration! `#CSharp` `#DotNet` `#OOP` `#ProgrammingTips`

2- What we mean by Generalization concept using Generics?

A: Generalization using Generics means writing a single algorithmic template that works securely with *any* data type. Instead of writing specific code for integers, and then duplicating it for strings, Generics (`<T>`) allow the compiler to dynamically substitute the required data type at compile-time. This provides type-safety, avoids boxing/unboxing performance hits, and drastically reduces code duplication.

3- What we mean by hierarchy design in real business?

A: In real business applications, hierarchy design represents a parent-child relationship structured to share common behavior and reduce repetition. For example, in a banking system, you might have a base class `BankAccount` with shared logic (Balance, Deposit), and specialized child classes like `SavingsAccount` and `CheckingAccount` that inherit the base logic but add specific business rules (Interest Rate, Overdraft Limits).